

1. 论文名称: Retrieval-Augmented Generation for Large Language Models: A Survey
2. github: <https://github.com/Tongji-KGLLM/RAG-Survey.git>

Abstract

摘要主要介绍了大型语言模型（LLMs）在实际应用中面临的挑战，比如幻觉、知识更新缓慢和答案缺乏透明度等问题，并提出了检索增强生成（Retrieval-Augmented Generation, RAG）作为一种解决方案。RAG通过从外部知识库检索相关信息来辅助大型语言模型回答问题，已经被证明能显著提高回答的准确性，减少模型产生的幻觉，尤其是在知识密集型任务中。

RAG的核心优势包括：

1. **提高答案准确性**：通过引用外部知识库中的信息，RAG可以提供更准确的回答。
2. **增加用户信任**：用户可以通过引用的来源来验证答案的准确性。
3. **便于知识更新和引入特定领域知识**：RAG通过结合LLMs的参数化知识和外部知识库的非参数化知识，有效地解决了知识更新的问题。

论文概述了RAG在大型语言模型时代的发展范式，总结了三种RAG范式：原始RAG（Naive RAG）、高级RAG（Advanced RAG）和模块化RAG（Modular RAG）。此外，还总结和组织了RAG的三个主要组成部分：检索器、生成器和增强方法，以及每个组件中的关键技术。此外，论文还讨论了如何评估RAG模型的有效性，介绍了两种评估RAG的方法，强调了评估的关键指标和能力，并提出了最新的自动评估框架。

最后，论文介绍了未来研究方向的潜力，从三个方面进行介绍：垂直优化、水平扩展性以及RAG的技术栈和生态系统。

1 Introduction

这篇论文的第一章引言部分深入探讨了大型语言模型（LLMs）在自然语言处理（NLP）领域的显著进步及其面临的挑战。主要内容可以概括为以下几个要点：

1.1 大型语言模型的进步

1. **模型发展**：提到了GPT系列、LLama系列、Gemini等大型语言模型，它们在多个评估基准测试中显示出超越人类水平的语言和知识掌握能力。
2. **模型性能**：这些模型在各种语言任务上展现出令人印象深刻的性能。

1.2 面临的挑战

1. **制造事实**：LLMs经常制造事实，特别是在处理特定领域或高度专业化的查询时。
2. **知识限制**：当所寻求的信息超出模型训练数据范围或需要最新数据时，LLMs可能无法提供准确答案。
3. **实际应用中的挑战**：在实际生产环境中部署生成型人工智能时，单纯使用“黑盒”LLMs可能不够。

1.3 解决方案：检索增强生成（RAG）

- 参数化和非参数化知识：**参数化知识是通过训练LLMs获得并储存在神经网络权重中的，而非参数化知识存在于外部知识源中，如向量数据库。
- RAG的定义和作用：**RAG是一种半参数化方法，结合了非参数化的语料库数据库和参数化模型。RAG由Lewis等人于2020年首次引入，结合了预训练的检索器和生成器，并进行端到端的微调，以更可解释、模块化的方式捕获知识。
- RAG的优势：**RAG通过从外部知识源动态检索信息，并使用检索到的数据作为组织答案的参考，显著提高了响应的准确性和相关性，有效解决了LLMs中存在的幻觉问题。

1.4 RAG的研究和发展

- RAG研究的演变：**文章概述了RAG的三种主要范式：原始RAG、高级RAG和模块化RAG，并总结了三个核心组成部分：检索、增强和生成。
- 未来的发展趋势：**提出了对RAG未来发展趋势的展望，包括评价体系、适用场景等相关内容。

1.5 研究背景

- 相关工作对比：**与之前的工作相比，本文旨在系统性地概述检索增强生成（RAG）的整个过程，专注于通过知识检索增强大型语言模型的生成。
- 研究时间线：**从2020年开始，尤其是在2022年12月ChatGPT发布后，RAG的研究取得了显著的进步。

2 Background

“背景”部分详细介绍了检索增强生成（Retrieval-Augmented Generation, RAG）的定义及其与其他模型优化技术（如微调）的对比。主要内容可以概括为以下几个要点：

2.1 RAG的定义

- 技术含义：**RAG的概念随技术发展而扩展。在大型语言模型时代，RAG指的是在回答问题或生成文本时，先从大规模文档库中检索相关信息，然后利用这些检索到的信息来生成响应或文本，从而提高预测的质量。
- RAG的两个关键阶段：**
 - 检索阶段：**使用编码模型（如BM25、DPR、ColBERT等）基于问题检索相关文档。
 - 生成阶段：**使用检索到的上下文作为条件生成文本。

2.2 RAG vs 微调

- 优化技术的差异：**RAG类似于为模型提供教科书，使其能够根据特定查询检索信息。适用于模型需要回答特定查询或处理特定信息检索任务的场景。微调则类似于让学生通过广泛学习来内化知识，适用于模型需要复制特定结构、风格或格式的情况。
- 适用场景：**
 - RAG：**适用于需要回答具体查询或特定信息检索任务的场景。
 - 微调：**适用于强调基础模型中的现有知识，修改或定制模型输出，为模型提供复杂指令的场景。

3. **互补性**：RAG和微调不是相互排斥的，而是可以相互补充，从不同层面增强模型的能力。在某些情况下，结合这两种技术可以实现最佳的模型性能。

2.3 RAG的优势

1. **准确性提高**：RAG通过与外部知识相关联的答案来提高准确性，减少语言模型中的幻觉问题，使生成的响应更准确、可靠。
2. **保持信息的时效性和准确性**：与传统只依赖训练数据的语言模型相比，RAG可以识别最新信息，保持响应的时效性和准确性。
3. **透明度**：RAG通过引用来源提高答案的透明度，增加用户对模型输出的信任。
4. **定制化能力**：RAG可以通过索引相关文本语料库来定制不同领域的模型，为特定领域提供知识支持。
5. **安全性和隐私管理**：RAG在数据库中内置了角色和安全控制，可以更好地控制数据使用。
6. **可扩展性**：RAG能够处理大规模数据集而无需更新所有参数和创建训练集，使其在经济上更有效率。
7. **结果的可信度**：RAG从最新数据中选择确定性结果，而微调模型在处理动态数据时可能出现幻觉和不准确性，缺乏透明度和可信度。

3 RAG Framework

“RAG框架”详细探讨了检索增强生成（RAG）的研究范式及其演变。这一章节将RAG分为三种类型：原始RAG（Naive RAG）、高级RAG（Advanced RAG）和模块化RAG（Modular RAG），并详细介绍了每种类型的特点和挑战。

3.1 原始RAG（Naive RAG）

原始RAG是最早的研究范式，主要包括以下几个步骤：

1. **建立索引**：这一过程通常在离线状态下进行，包括数据清理、提取，将不同文件格式（如PDF、HTML、Word、Markdown等）转换为纯文本，然后进行文本分块，并创建索引。
2. **检索**：使用相同的编码模型将用户输入转换为向量，计算问题嵌入和文档块嵌入之间的相似度，选择相似度最高的前K个文档块作为当前问题的增强上下文信息。
3. **生成**：将给定的问题和相关文档合并为新的提示，然后由大型语言模型基于提供的信息回答问题。如果有历史对话信息，也可以合并到提示中，用于多轮对话。

原始RAG面临的挑战

- **检索质量问题**：包括低精度（检索集中不所有块都与查询相关，导致可能的幻觉和断层问题）和低召回率（无法检索到所有相关块，阻止LLM获取足够的上下文来合成答案）等。
- **响应生成质量问题**：包括幻觉（模型编造不存在于上下文中的答案）、不相关性（模型生成的答案未能解决查询问题）以及有害或偏见性回应等。
- **增强过程的挑战**：包括有效整合检索段落的上下文与当前生成任务、冗余和重复（多个检索段落包含相似信息导致内容重复）、确定多个检索段落对生成任务的重要性或相关性、处理不同写作风格或语气的差异以确保输出一致性等。

3.2 高级RAG (Advanced RAG)

高级RAG针对原始RAG的不足之处进行了有针对性的改进。这些改进涉及检索生成的质量、索引的优化、检索过程的优化等多个方面。

3.2.1 预检索过程

- **优化数据索引**：提高索引内容的质量，包括增加索引数据的细粒度、优化索引结构、添加元数据、对齐优化和混合检索等五个主要策略。
- **嵌入和索引**：涉及通过语言模型将文本编码为向量的过程，包括微调嵌入和动态嵌入。
 - **微调嵌入**：提高检索内容与查询之间的相关性。
 - **动态嵌入**：基于单词出现的上下文动态调整嵌入。

3.2.2 检索过程优化

- **重新排序**：对检索到的信息进行重新排序，将最相关的信息放置在提示的边缘。
- **提示压缩**：通过压缩不相关的上下文、突出关键段落和减少总体上下文长度来提高RAG性能。

3.2.3 高级RAG的特点

- 高级RAG通过先进的索引方法、更精细的检索策略和有效的后处理方法来优化RAG的性能。
- 它采用不同的检索技术组合，适应不同的查询类型和信息需求，确保一致地检索到最相关和上下文丰富的信息。
- 高级RAG还包括对嵌入模型的微调，以提高检索内容的相关性，并使用动态嵌入技术以更好地处理上下文变化。

3.2.4 RAG管道优化

在“RAG管道优化”这一部分中，论文讨论了针对检索过程的优化，旨在提高RAG系统的效率和信息质量。这些研究主要集中在智能组合各种搜索技术、优化检索步骤、引入认知回溯的概念、灵活应用多样化的查询策略，以及利用嵌入相似性。这些努力共同追求在RAG检索中实现效率和上下文信息丰富度之间的平衡。

RAG检索过程的优化方法：

1. **探索混合搜索**：通过智能融合关键字搜索、语义搜索和向量搜索等技术，RAG系统可以利用每种方法的优势，适应不同类型的查询和信息需求，确保一致地检索到最相关和上下文丰富的信息。
2. **递归检索和查询引擎**：实施递归检索和复杂查询引擎是优化RAG系统检索的另一种强有力的方法。递归检索意味着在初始检索阶段获取较小的文档块以捕获关键语义含义，然后在后续阶段向语言模型提供更多上下文信息的较大块。这种两步检索方法有助于在效率和富有上下文的响应之间找到平衡。
3. **StepBack-prompt**：将StepBack-prompt方法集成到RAG过程中，鼓励大型语言模型从特定实例中退后，参与对背后的一般概念或原则的推理。实验结果表明，在各种具有挑战性的、推理密集的任务中，结合后向提示的运用能显著提高性能，显示出其在RAG中的自然适应性。
4. **子查询**：可以在不同场景中采用各种查询策略，包括使用LlamaIndex等框架提供的查询引擎，运用树查询、向量查询，或使用最基本的块序列查询。

5. **HyDE**：这种方法基于假设，即生成的答案在嵌入空间中可能比直接查询更接近。使用大型语言模型，HyDE针对查询生成一个假设的文档（答案），嵌入该文档，并利用这种嵌入来检索与假设文档类似的真实文档。与基于查询的嵌入相似性不同，这种方法强调从答案到答案的嵌入相似性。然而，这种方法可能并不总是能够产生有利的结果，特别是在语言模型不熟悉讨论主题的情况下，可能导致生成更多错误实例。

3.3 模块化RAG（Modular RAG）

模块化RAG结构打破了传统的原始RAG框架（索引、检索和生成），提供了更大的多样性和整个过程的灵活性。它整合了各种方法来扩展功能模块，例如在相似性检索中加入搜索模块，以及在检索器中应用微调方法。此外，特定问题的出现促使重构的RAG模块和迭代方法的出现。模块化RAG范式正成为RAG领域的主流，允许采用序列化管道或跨多个模块的端到端训练方法。

3.3.1 新模块

1. **搜索模块**：与原始/高级RAG中的查询和语料库之间的相似性检索不同，搜索模块针对特定场景，将直接搜索引入过程中，使用由LLM生成的代码、查询语言（如SQL、Cypher）或其他自定义工具。
2. **记忆模块**：利用LLM自身的记忆能力来指导检索。原则是找到与当前输入最相似的记忆。例如，Self-mem迭代地使用一个检索增强生成器来创建一个无限的记忆池，结合“原始问题”和“对偶问题”。
3. **额外生成模块**：在检索到的内容中，冗余和噪声是常见问题。额外生成模块利用LLM生成所需的上下文，而不是直接从数据源检索。
4. **任务适应模块**：专注于转换RAG以适应各种下游任务。例如，UPRISE自动从预构建的数据池中检索给定零样本任务输入的提示，增强跨任务和模型的通用性。
5. **对齐模块**：查询和文本之间的对齐一直是影响RAG有效性的关键问题。在模块化RAG时代，研究人员发现，在检索器中添加一个可训练的Adapter模块可以有效缓解对齐问题。
6. **验证模块**：在现实世界场景中，不能总保证检索到的信息是可靠的。检索到不相关的数据可能导致LLM出现幻觉。因此，在检索文档后可以引入额外的验证模块，以评估检索到的文档与查询之间的相关性，增强RAG的鲁棒性。

3.3.2 新模式

模块化RAG的组织方法具有灵活性，允许根据具体问题上下文替换或重新配置RAG过程中的模块。对于原始RAG而言，这种框架提供了适应性和丰富性。当前的研究主要探索两种组织范式，包括添加或替换模块以及调整模块之间的组织流程。

（1）添加或替换模块

- **模块添加或替换策略**：维持检索-阅读的结构，同时引入额外的模块来增强特定功能。例如，RRR[Ma et al., 2023a] 提出了重写-检索-阅读过程，使用LLM性能作为强化学习中重写器模块的奖励。这允许重写器调整检索查询，提高阅读器的下游任务性能。类似地，可以在Generate-Read[Yu et al., 2022]方法中选择性地替换模块，其中LLM生成模块替换了检索模块。Recite-Read [Sun et al., 2022] 将外部检索转换为从模型权重中检索，让LLM先记住任务相关信息，再生成输出，以处理知识密集型自然语言处理任务。

(2) 调整模块间流程

- **模块间流程调整**：强调增强语言模型与检索模型之间的互动。DSP[Khattab et al., 2022] 引入了展示-搜索-预测框架，将上下文学习系统视为显式程序而不是终端任务提示，以解决知识密集型任务。ITER-RETGEN [Shao et al., 2023] 利用生成内容指导检索，迭代执行“检索增强生成”和“生成增强检索”，形成检索-阅读-检索-阅读的流程。Self-RAG[Asai et al., 2023b] 遵循决定-检索-反思-阅读过程，引入了一个用于主动判断的模块。这种自适应和多样化的方法允许在模块化RAG框架内动态组织模块。

4 Retriever 检索器

在RAG（检索增强生成）框架中，“R”代表检索。它在RAG管道中的作用是从庞大的知识库中检索出最相关的前k个文档。然而，构建一个高质量的检索器并非易事。本章围绕三个关键问题进行讨论：1) 如何获得准确的语义表示？2) 如何匹配查询和文档的语义空间？3) 如何使检索器的输出与大型语言模型的偏好相一致？

4.1 如何获得准确的语义表示？

在RAG中，语义空间是将查询和文档映射到的多维空间。执行检索时，需要在语义空间内进行测量。如果语义表达不准确，那么它对RAG的影响是致命的。这部分将介绍两种帮助我们构建准确语义空间的方法。

4.1.1 块优化

- 在处理外部文档时，第一步是进行分块，以获得细粒度特征，然后将这些块进行嵌入。然而，嵌入过大或过小的文本块可能无法获得好的结果。因此，找到语料库中文档的最佳块大小对于确保搜索结果的准确性和相关性至关重要。
- 在选择分块策略时，需要考虑的因素包括：被索引内容的特点、所使用的嵌入模型及其最佳块大小、用户查询的预期长度和复杂度，以及特定应用中检索结果的使用方式。

4.1.2 微调嵌入模型

- 在获取适当大小的块之后，我们需要通过嵌入模型将块和查询嵌入到语义空间中。优秀的嵌入模型如UAE、Voyage、BGE等已经在大规模语料库上进行了预训练，但在特定领域中可能无法准确表示领域特定的语料库信息。
- 针对特定任务的嵌入模型微调对于确保模型理解用户查询与内容相关性至关重要。微调的嵌入模型对于下游应用是必不可少的。
- 领域知识微调：为了使嵌入模型正确理解领域特定信息，我们需要构建领域特定数据集来微调嵌入模型。
- 下游任务的微调：调整嵌入模型以适应下游任务同样重要。当在下游任务中使用RAG时，一些工作通过利用LLMs的能力对嵌入模型进行了微调。

4.2 如何匹配查询和文档的语义空间

在RAG应用中，一些检索器使用相同的嵌入模型来编码查询和文档，而其他检索器则使用两个模型分别对查询和文档进行编码。此外，用户的原始查询可能存在表达不佳和缺乏语义信息的问题。因此，使用户查询和文档的语义空间对齐非常必要。本节介绍两项关键技术以实现这一目标。

4.2.1 查询重写

- 将查询和文档的语义对齐最直观的方式是重写查询。如Query2Doc[Wang et al., 2023b]和ITER-RETGEN[Shao et al., 2023]所提到的，利用大型语言模型的内在能力生成伪文档，并将原始查询与这个伪文档合并形成新的查询。在HyDE[Gao et al., 2022]中，通过使用文本指示符建立查询向量，使用这些指示符生成一个相关但可能并不存在的“假设”文档，它只需要捕获相关模式。RRR[Ma et al., 2023a]引入了一种新框架，将检索和阅读的顺序颠倒，专注于查询重写。此方法使用大型语言模型生成查询，然后使用网络搜索引擎检索上下文，最后使用小型语言模型作为训练重写器服务于冻结的大型语言模型。STEP-BACKPROMPTING[Zheng et al., 2023]方法可以使大型语言模型进行抽象推理，提取高层次概念和原则，并基于此进行检索。最后，Multi Query Retrieval方法涉及使用大型语言模型生成多个搜索查询，这些查询可以并行执行，检索结果一起输入，这对于依赖多个子问题的单一问题非常有用。

4.2.2 嵌入变换

- 如果有像重写查询这样的粗粒度方法，那么也应该有针对嵌入操作的更细粒度实现。在LlamaIndex[Liu, 2023]中，可以在查询编码器后连接一个适配器，并微调适配器以优化查询嵌入的表示，将其映射到更适合特定任务的潜在空间。当查询和外部文档的数据结构不同时，如非结构化查询和结构化外部文档，使查询与文档对齐非常重要。SANTA[Li et al., 2023d]提出了两种预训练方法，使检索器意识到结构化信息：1) 使用结构化数据和非结构化数据之间的自然对齐关系进行对比学习，以进行结构化意识的预训练。2) Masked Entity Prediction，设计了一种以实体为中心的遮蔽策略，并要求语言模型填充被遮蔽的实体。

4.3 如何使检索器的输出与LLM的偏好一致

即使我们采用了上述技术来提高检索命中率，RAG的最终效果可能仍然无法提升，因为检索到的文档可能不是LLM所需要的。因此，本节介绍了两种方法来使检索器的输出与LLM的偏好对齐。

4.3.1 LLM监督训练

- 许多研究利用大型语言模型的不同反馈信号来微调嵌入模型。AAR[Yu et al., 2023b]通过编码器-解码器架构的LM为预训练检索器提供监督信号。通过确定LM偏好的文档，并利用FiD跨注意力得分进行微调检索器，然后使用硬负采样和标准交叉熵损失。最终，微调后的检索器可以直接用于增强未见过的目标LM，从而在目标任务中表现更好。
- REPLUG[Shi et al., 2023]使用检索器和LLM计算检索到的文档的概率分布，然后通过计算KL散度进行监督训练。这种简单有效的训练方法通过使用LM作为监督信号来提升检索模型的性能，无需任何特定的跨注意力机制。

4.3.2 插入适配器

- 由于使用API实现嵌入功能或本地计算资源不足等因素，微调嵌入模型可能具有挑战性。因此，一些研究选择外部附加适配器进行对齐。PRCA[Yang et al., 2023b]通过上下文提取阶段和奖励驱动阶段训练适配器，并根据基于token的自回归策略优化检索器的输出。
- TokenFiltering[Berchansky et al., 2023]方法计算跨注意力得分，选择得分最高的输入token来有效过滤token。RECOMP[Xu et al., 2023a]提出了抽取和生成压缩器，通过选择相关句子或合成文档信息来生成摘要，实现多文档查询重点摘要。此外，一种新方法PKG[Luo et al., 2023]通过指令微调将知识注入白盒模型，并直接替换检索器模块，用于直接根据查询输出相关文档。

5 Generator 生成器

在RAG中，生成器是另一个核心组件，负责将检索到的信息转换成自然流畅的文本。与传统的语言模型相比，RAG的生成器通过利用检索到的信息来提高准确性和相关性。在RAG中，生成器的输入不仅包括传统的上下文信息，还包括通过检索器获得的相关文本段落。这使得生成器能够更好地理解问题背后的上下文，并产生更丰富信息的回应。

5.1 如何通过后检索处理增强检索结果？

即使使用了上述技术来提高检索命中率，大型未调整的语言模型（如GPT-4）仍然依赖于其强大的内部知识来全面检索文档知识。然而，这些大型模型的固有问题，如上下文长度限制和对冗余信息的易受影响，仍然存在。为了解决这些问题，一些研究在后检索处理方面做出了努力。后检索处理指的是进一步处理、过滤或优化检索器从大型文档数据库检索到的相关信息的过程。其主要目的是提高检索结果的质量，以更好地满足用户需求或后续任务。它可以被理解为对检索阶段获得的文档进行再处理。

5.1.1 信息压缩

- 即使检索器可以从庞大的知识库中获取相关信息，我们仍然面临处理检索文档中大量信息的挑战。一些现有研究尝试通过增加大型语言模型的上下文长度来解决这个问题，但当前的大型模型仍然面临上下文限制。因此，在某些情况下，信息压缩是必要的。信息压缩的重要性主要体现在减少噪声、应对上下文长度限制和增强生成效果方面。

5.1.2 重排

- 文档集重排模型的关键作用在于优化检索器检索到的文档集。LLM在增加额外上下文时性能下降，重排提供了解决这个问题的有效方法。核心思想是重新排列文档记录，将最相关的项目放在顶部，从而将文档总数减少到固定数量。这不仅解决了检索过程中可能遇到的上下文窗口扩展问题，还有助于提高检索效率和响应能力。

5.2 如何优化生成器以适应输入数据？

在RAG模型中，生成器的优化是架构的关键组成部分。生成器的任务是将检索到的信息转换成相关文本，从而提供模型的最终输出。优化生成器的目标是确保生成的文本既自然又有效地利用检索到的文档，以更好地满足用户的查询需求。

在大型语言模型（LLM）的生成任务中，输入通常是一个查询。在RAG中，主要区别在于输入不仅包括查询，还包括检索器检索到的各种文档（结构化/非结构化）。引入额外信息可能会显著影响模型的理解，尤其对于较小的模型。在这种情况下，微调模型以适应查询+检索文档的输入变得尤为重要。具体来说，在将输入提供给微调模型之前，通常需要对检索器检索到的文档进行后检索处理。需要注意的是，RAG中生成器的微调方法本质上类似于LLM的一般微调方法。以下是一些代表性的工作，包括数据（格式化/非格式化）和优化功能的简要介绍。

5.2.1 一般优化过程

- 指的是包含（输入，输出）对的训练数据，旨在训练模型在给定输入x的情况下生成输出y的能力。在Self-mem[Cheng et al., 2023b]的工作中，采用了一种相对经典的训练过程。给定输入x，检索相关文档z（论文中选择Top-1），整合（x, z）后，模型生成输出y。论文采用了两种常见的微调范式，即联合编码器[Arora et al., 2023, Wang et al., 2022b, Lewis et al., 2020]和双编码器[Xia et al., 2019, Cai et al., 2021, Cheng et al., 2022]。

5.2.2 利用对比学习

- 在准备训练数据的阶段，通常生成输入和输出之间的交互对。在这种情况下，模型只能访问唯一真实的输出，可能会引起“暴露偏差”问题[Ranzato et al., 2015]：在训练阶段，模型只暴露于单个真实反馈，无法访问任何其他生成的token。这可能会损害模型在应用中的性能，因为它可能过度适应训练数据中的特定反馈，而无法有效地泛化到其他场景。因此，SURGE[Kang et al., 2023]提出了一种图文对比学习方法。

5.2.3 优化目标

- 在处理涉及结构化数据的检索任务时，SANTA[Li et al., 2023d]使用了三阶段训练过程来充分理解结构和语义信息。在检索器的训练阶段，采用了对比学习，主要目标是优化查询和文档的嵌入表示。

6 Augmentation in RAG

本章主要围绕三个维度组织内容：增强阶段、增强数据来源和增强过程，旨在详细阐述RAG发展中的关键技术。

6.1 RAG的增强阶段

作为一项知识密集型任务，RAG在语言模型训练的预训练、微调和推理阶段采用了不同的技术方法。

6.1.1 预训练阶段

这段论文讨论了在预训练阶段使用检索方法来增强预训练语言模型（PTMs）在开放域问答（QA）中的性能。这里主要介绍了几种不同的方法和模型，包括REALM、RETRO、Atla和COG，以及RETRO++。下面我将对每个模型的核心思想和特点进行概述和分析。

1. REALM (Arora et al., 2023) :

- 目标：提出一种更模块化、可解释的知识嵌入方法。
- 方法：遵循掩码语言模型（MLM）范式，将预训练和微调视为一个检索-然后-预测的过程。在预训练阶段，模型通过预测掩码句子x中的掩码词汇y来进行学习，模拟 $P(x|y)$ 。

2. RETRO (Borgeaud et al., 2022) :

- 目标：利用检索增强来预训练一个自回归语言模型。
- 方法：从大量标记数据中检索，大幅减少模型参数。RETRO在GPT模型的基础上增加了一个额外的RETRO编码器，用于编码从外部知识库检索到的邻近实体的特征，并在解码器结构中加入块状交叉注意层以有效整合检索信息。RETRO在多方面性能上优于标准GPT模型。

3. **Atla** (Izacard et al., 2022) :

- 目标：使用T5架构，在预训练和微调阶段引入检索机制。
- 方法：预训练前，使用预训练的T5初始化编码器-解码器LM骨架，使用预训练的Contriever初始化密集检索器。在预训练过程中，每1000步刷新一次异步索引。

4. **COG** (Vaze et al., 2021) :

- 目标：通过逐渐从现有文本集合中复制文本片段（如单词或短语）来形式化其生成过程。
- 方法：与传统文本生成模型不同，COG使用高效的向量搜索工具来计算文本片段的有意义的上下文表示，并对其进行索引。因此，文本生成任务被分解为一系列的复制和粘贴操作。

5. **RETRO++** (Wang et al., 2023a) :

- 目标：增加模型的参数规模，探索大型模型是否可以使用RAG方法进行预训练。
- 特点：在文本生成质量、事实准确性、低毒性和下游任务准确性方面取得了一致的改进，特别是在知识密集型任务中表现突出。

6.1.2 微调阶段

在微调阶段，研究者们采用了多种方法来微调检索器和生成器，以提高信息检索的效果，主要应用于开放域问答任务。下面是对这些方法的概述和分析：

1. **REPLUG** (Shi et al., 2023) :

- 目标：将语言模型（LM）视为黑盒，通过可调整的检索模型来增强它。
- 方法：通过从黑盒语言模型获得的监督信号，REPLUG改进了初始检索模型。

2. **UPRISE** (Cheng et al., 2023a) :

- 目标：通过在多样化的任务集上微调来创建轻量级和多功能的检索器。
- 特点：这种检索器能自动为零样本任务提供检索提示，显示其普遍性和在任务和模型间的改进性能。

3. 微调生成器的方法:

- **Self-Mem** (Cheng et al., 2023b) : 通过示例的内存池来微调生成器。
- **Self-RAG** (Asai et al., 2023b) : 通过生成反射令牌来满足主动检索需求。
- **RADIT** (Lin et al., 2023) : 通过最大化给定检索增强指令的正确答案的概率来微调生成器和检索器。RADIT更新生成器和检索器，以最小化文档和查询之间的语义相似性。

4. **SURGE** (Kang et al., 2023) :

- 目标：引入对比学习的概念，对检索器和生成器进行端到端微调。
- 方法：使用基于图神经网络（GNN）的上下文感知子图检索器，从与正在进行的对话相关的知识图中提取相关知识。

微调阶段的增强方法具有几个特点。首先，同时微调LLM和检索器可以更好地适应特定任务，提供了灵活性，可以单独或同时微调两者。其次，这种微调的好处还扩展到适应多样化的下游任务，使模型更加多功能。此外，微调还使模型能更好地适应不同语料库中的不同数据结构，特别是对于图结构语料库，如SURGE方法所示。

然而，这一阶段的微调也有局限性，如需要为RAG微调专门准备的数据集，以及与推理阶段的RAG相比需要大量的计算资源。总的来说，在微调过程中，研究者可以根据特定的需求和数据格式定制模型，与预训练阶段相比，这降低了资源消耗，同时保留了调整模型输出风格的能力。

6.1.3 推理阶段

在推理阶段，将检索-生成增强（RAG）方法与大型语言模型（LLM）结合成为了一个流行的研究方向。具体来说，这个方向主要依赖于在推理阶段整合检索内容。以下是对这方向中几种方法的概述和分析：

1. Naive RAG:

- 特点：在推理阶段整合检索内容。

2. DSP (Khattab et al., 2022) :

- 目标：在RAG中引入更丰富的上下文。
- 方法：依赖于一个复杂的流程，涉及在固定的语言模型（LM）和检索模型（RM）之间传递自然语言文本，为模型提供更有信息的上下文以增强生成质量。

3. PKG:

- 目标：为LLMs配备一个知识引导模块，允许访问相关知识而不改变LLMs的参数。
- 优点：使模型能执行更复杂的任务。

4. CREA-ICL (Li et al., 2023b) :

- 方法：利用同步检索跨语言知识来帮助获取额外信息。

5. RECITE:

- 方法：通过从LLMs中抽取一个或多个段落来形成上下文。

6. ITRG (Feng et al., 2023a) :

- 目标：增强对需要多步推理的任务的适应性。
- 方法：通过迭代检索和搜索正确的推理路径来实现。

7. ITERRETGEN (Shao et al., 2023) :

- 方法：采用迭代方法来整合检索和生成，实现“检索增强生成”和“生成增强检索”的交替过程。

8. IRCOT (Trivedi et al., 2022) :

- 方法：融合RAG和CoT (Wei et al., 2022) 的概念，使用交替的CoT引导检索并使用检索结果来改善CoT。
- 优点：显著提高了GPT-3在各种QA任务上的性能。

6.2 数据源增强

数据源是RAG（检索-生成增强）方法有效性的关键因素。不同的数据源提供了不同的知识粒度和维度，需要不同的处理方法。它们主要分为三类：非结构化数据、结构化数据和由大型语言模型（LLM）生成的内容。

6.2.1 非结构化数据的增强

非结构化数据主要包括文本数据，通常来自纯文本语料库。此外，其他文本数据也可以作为检索源，例如用于大型模型微调的提示数据[Cheng et al., 2023a]和跨语言数据[Li et al., 2023b]。

- 文本粒度方面，除了常见的块（包括句子），检索单元可以是词汇（例如，kNN-LM[Khandelwal et al., 2019]）、短语（例如，NPM[Lee et al., 2020], COG[Vaze et al., 2021]）和文档段落。更细粒度的检索单元通常能更好地处理罕见模式和域外场景，但增加了检索成本。
- 在词汇层面，FLARE采用主动检索策略，仅在LM生成低概率词汇时进行检索。该方法涉及生成临时的下一句话以检索相关文档，然后在检索文档的条件下重新生成下一句话，以预测后续句子。
- 在块层面，RETRO使用前一个块来检索最近邻的块，并将此信息与前一个块的上下文信息整合，以指导下一个块的生成。RETRO通过交叉注意力将前面的块（ $C_1, \dots, C_{i-1}$ ）的上下文信息和最近邻块 $N(C_{i-1})$ 的检索信息融合，以指导下一个块 C_i 的生成。

6.2.2 结构化数据的增强

结构化数据源，如知识图谱（KG），正逐渐融入RAG的范式。经过验证的KG可以提供更高质量的上下文，降低模型幻觉的可能性。

- RET-LLM [Modarressi et al., 2023] 通过从过去的对话中提取关系三元组来构建个性化的知识图谱记忆，供将来使用。
- SUGRE[Kang et al., 2023] 使用图神经网络（GNN）嵌入从知识图谱中检索的相关子图，防止模型生成上下文不相关的回复。SUGRE采用了一种图编码方法，将图结构反映到PTMs的表示空间中，并使用图文模式间的多模态对比学习目标，以确保检索到的事实与生成的文本之间的一致性。
- KnowledgeGPT[Wang et al., 2023c] 以代码格式生成知识库（KB）的搜索查询，并包含预定义的KB操作函数。除了检索外，KnowledgeGPT还提供了在个性化知识库中存储知识的能力，以满足个别用户的需求。

6.2.3 LLM生成内容的RAG

有些研究发现，RAG回忆的辅助信息并不总是有效的，甚至可能有负面效果，因此拓展了RAG的范式，更深入地挖掘LLM的内部知识。这种方法利用LLM自身生成的内容进行检索，旨在提高下游任务中的性能。以下是此类别中的一些显著研究：

- SKR[Wang et al., 2023d] 使用标记的训练集，将模型可以直接回答的问题分类为已知，需要检索增强的分类为未知。模型被训练以辨别问题是否已知，仅将检索增强应用于识别为未知的输入，而直接回答其他问题。
- GenRead[Yu et al., 2022] 将LLM生成器替换为检索器。实验结果表明，生成的上下文文档包含正确答案的情况比Naive RAG检索到的情况更普遍。生成的答案也展示出更高的质量。作者将这归因于生成文档级上下文的任务与因果语言建模的预训练目标之间的一致性，允许更好地利用模型参数中存储的世界知识。
- Selfmem[Cheng et al., 2023b] 迭代使用检索增强生成器来创建一个无界的内存池。使用内存选择器选择输出作为后续生成的内存。这个输出作为原始问题的对偶问题。通过结合原始问题和对偶问题，检索增强的生成模型可以利用自己的输出来增强自己。

这些多样化的方法展示了RAG检索增强的创新策略，旨在提升模型性能和有效性。

6.3 过程增强

RAG（检索-生成增强）研究中，大多数情况下仅执行一次检索和生成过程。然而，单次检索可能包含冗余信息，导致“中间迷失”现象[Liu et al., 2023]。这种冗余信息可能会掩盖关键信息或包含与真实答案相反的信息，从而对生成效果产生负面影响[Yoran et al., 2023]。此外，单次检索获得的信息在需要多步推理的问题上是有限的。

为优化检索过程，当前方法主要包括迭代检索和自适应检索。这些方法允许模型在检索过程中多次迭代或自适应地调整检索过程，以更好地适应不同任务和场景。

6.3.1 迭代检索

- 根据原始查询和生成的文本定期收集文档可以为LLM提供额外的材料[Borgeaud et al., 2022, Arora et al., 2023]。多次迭代检索中提供额外参考资料已提高了后续答案生成的鲁棒性。然而，这种方法可能在语义上不连贯，并可能导致收集噪声和无用信息，因为它主要依赖于一系列n个令牌来分离生成和检索的文档。
- 递归检索和多跳检索用于特定的数据场景。递归检索可以首先通过结构化索引处理数据，然后逐级检索。在检索层次丰富的文档时，可以为整个文档或长PDF的每个部分制作摘要，然后根据摘要进行检索。确定文档后，对内部块进行第二次检索，从而实现递归检索。多跳检索通常用于进一步挖掘图结构数据源中的信息[Li et al., 2023c]。

6.3.2 自适应检索

- RAG方法中描述的前两个部分遵循的是一种被动方法，即优先考虑检索。这种方法，涉及根据上下文查询相关文档并输入到LLM中，可能导致效率问题。自适应检索方法，如Flare[Jiang et al., 2023b]和Self-RAG[Asai et al., 2023b]，优化了RAG检索过程，使LLM能够主动判断检索的时机和内容，帮助提高检索信息的效率和相关性。
- 实际上，LLM主动使用工具和做出判断的方式并非起源于RAG，而是已在大型模型的代理中广泛使用[Yang et al., 2023c, Schick et al., 2023, Zhang, 2023]。Graph-Toolformer[Zhang, 2023]的检索步骤大致分为：LLMs主动使用检索器，Self-Ask和DSP[Khattab et al., 2022]尝试使用小样本提示触发LLM搜索查询。当LLM认为有必要时，可以决定搜索相关查询以收集必要材料，类似于代理的工具调用。

这些不同的方法展示了在RAG检索增强中的创新策略，旨在提升模型性能和有效性。通过迭代检索和自适应检索的应用，LLM可以更有效地处理复杂和多维度的问题，同时避免无效或不相关信息的干扰，从而提高生成内容的准确性和相关性。

7 RAG Evaluation

在探索检索-生成增强（RAG）的发展和优化中，有效评估其性能成为一个核心问题。本章主要讨论评估方法、RAG的关键指标、它应具备的能力，以及一些主流的评估框架。

7.1 评估方法

评估RAG有效性主要有两种方法：独立评估和端到端评估[Liu, 2023]。

7.1.1 独立评估

独立评估包括对检索模块和生成（读取/合成）模块的评估。

1. 检索模块:

- 通常使用一套度量系统（如搜索引擎、推荐系统或信息检索系统）按照查询或任务对项目排名的有效性的指标来评估RAG检索模块的性能。例如，命中率、MRR、NDCG、精度等。

2. 生成模块:

- 这里的生成模块指的是通过将检索到的文档补充到查询中形成的增强或合成输入，与通常在端到端评估中评估的最终答案/响应生成不同。生成模块的评估指标主要关注上下文相关性，衡量检索文档与查询问题的相关性。

7.1.2 端到端评估

端到端评估评估RAG模型针对给定输入生成的最终响应，涉及模型生成答案与输入查询的相关性和一致性。从内容生成目标的角度来看，评估可以分为无标签内容和有标签内容。无标签内容评估指标包括答案的忠实度、相关性、无害性等，而有标签内容评估指标包括准确度和EM。此外，从评估方法的角度来看，端到端评估可以分为人工评估和使用LLM的自动评估。上述概括了RAG的端到端评估的一般情况。此外，根据RAG在特定领域的应用，采用特定的评估指标，例如问答任务中的EM[Borgeaud et al., 2022, Izacard et al., 2022]，总结任务中的UniEval和E-F1[jiang et al., 2023b]，以及机器翻译中的BLEU[Zhong et al., 2022]。这些指标有助于理解RAG在各种特定应用场景中的性能。

7.2 关键指标和能力

在现有的研究中，经常缺乏对检索增强生成（RAG）对不同大型语言模型（LLMs）影响的严格评估。在大多数情况下，对RAG在不同下游任务中的应用以及与不同检索器的结合可能产生不同的结果。然而，一些学术和工程实践已经关注了RAG的通用评估指标以及其有效使用所需的能力。本节主要介绍评估RAG效果的关键指标和评估其性能的基本能力。

7.2.1 关键指标

最近OpenAI的报告[jarvis and Allard, 2023]提到了用于优化大型语言模型（LLMs）的各种技术，包括RAG及其评估指标。此外，最新的评估框架，如RAGAS[Es et al., 2023]和ARES[Saad-Falcon et al., 2023]，也涉及RAG评估指标。综合这些工作，主要关注以下三个核心指标：答案的忠实度、答案相关性和上下文相关性。

1. 忠实度:

- 这个指标强调模型生成的答案必须保持对给定上下文的忠实，确保答案与上下文信息一致，不偏离或矛盾。这方面的评估对于解决大型模型中的幻觉问题至关重要。

2. 答案相关性:

- 这个指标强调生成的答案需要与提出的问题直接相关。

3. 上下文相关性:

- 这个指标要求检索到的上下文信息尽可能准确和针对性强，避免无关内容。毕竟，处理长文本对LLMs来说成本高昂，过多的无关信息会降低LLMs利用上下文的效率。

OpenAI的报告还提到了“上下文回忆”作为一个补充指标，用于衡量模型回答问题所需的所有相关信息的检索能力。这个指标反映了RAG检索模块的搜索优化水平。低回忆率表明可能需要优化搜索功能，如引入重新排序机制或微调嵌入，以确保检索到更相关的内容。

7.2.2 关键能力

1. 噪声鲁棒性:

- 这项能力衡量模型处理与问题相关但不包含有用信息的噪声文档的效率。

2. 负面拒绝:

- 当模型检索到的文档缺乏回答问题所需的知识时，模型应该正确地拒绝响应。在负面拒绝的测试设置中，外部文档只包含噪声。理想情况下，LLM应该发出“缺乏信息”或类似的拒绝信号。

3. 信息整合:

- 这项能力评估模型是否能从多个文档中整合信息以回答更复杂的问题。

4. 反事实鲁棒性:

- 这项测试旨在评估模型在接收到关于检索信息潜在风险的指令时，是否能识别并处理文档中已知的错误信息。反事实鲁棒性测试包括LLM可以直接回答的问题，但相关的外部文档包含事实错误。

这些能力的评估有助于理解RAG模型在处理复杂信息和不确定环境中的性能，从而为RAG的进一步发展和优化提供重要的参考依据。

7.3 评估框架

近年来，大型语言模型（LLM）社区一直在探索使用“LLM作为评判”进行自动评估的方法，许多研究者利用强大的LLM（如GPT-4）来评估其自身LLM应用的输出。Databricks使用GPT-3.5和GPT-4作为LLM评判来评估其聊天机器人应用表明，使用LLM作为自动评估工具是有效的[Leng et al., 2023]。他们认为这种方法也可以高效且成本效益地评估基于RAG的应用。

在RAG评估框架领域，RAGAS和ARES是相对较新的框架。这些评估的核心焦点是三个主要指标：答案的忠实度、答案相关性和上下文相关性。此外，由业界提出的开源库TruLens也提供了类似的评估模式。这些框架都使用LLM作为评判。由于TruLens与RAGAS类似，本章将具体介绍RAGAS和ARES。

7.3.1 RAGAS

RAGAS考虑了检索系统识别相关和关键上下文段落的能力，LLM忠实使用这些段落的能力，以及生成本身的质量。RAGAS是一个基于简单手写提示的评估框架，使用这些提示以完全自动化的方式衡量答案忠实度、答案相关性和上下文相关性的三个方面的质量。在这个框架的实施和实验中，所有提示都使用通过OpenAI API提供的gpt-3.5-turbo-16k模型进行评估[Es et al., 2023]。

<https://github.com/explodinggradients/ragas.git>

算法原理

1. **评估答案忠实度**：使用LLM将答案分解为单独的陈述，并验证每个陈述是否与上下文一致。最终，通过将支持的陈述数量与总陈述数量进行比较来计算“忠实度分数”。
2. **评估答案相关性**：使用LLM生成潜在问题，并计算这些问题与原始问题之间的相似度。“答案相关性得分”通过计算所有生成问题与原始问题的平均相似度而得出。
3. **评估上下文相关性**：使用LLM提取与问题直接相关的句子，并使用这些句子与上下文中总句子数量的比例作为“上下文相关性得分”。

7.3.2 ARES

ARES旨在自动评估RAG系统在三个方面的性能：上下文相关性、答案忠实度和答案相关性。这些评估指标与RAGAS中的类似。然而，RAGAS作为一个基于简单手写提示的较新评估框架，对新的RAG评估设置的适应性有限，这是ARES工作的一个重要意义。此外，如其评估所示，ARES的表现显著低于RAGAS。

ARES通过使用少量手动注释数据和合成数据来降低评估成本，并利用预测驱动推理（PDR）提供统计置信区间，增强评估的准确性[Saad-Falcon et al., 2023]。

```
https://github.com/stanford-futuredata/ARES.git
```

算法原理

1. **生成合成数据集**：ARES最初使用语言模型从目标语料库中的文档生成合成问题和答案，以创建正面和负面样本。
2. **准备LLM评判**：接下来，ARES使用合成数据集微调轻量级语言模型，训练它们评估上下文相关性、答案忠实度和答案相关性。
3. **使用置信区间排名RAG系统**：最后，ARES应用这些评判模型对RAG系统进行评分，并将它们与手动注释的验证集结合使用PPI方法生成置信区间，可靠地估计RAG系统的性能。

8 Future Prospects

本章我们深入探讨了RAG（检索-生成增强）的三个未来发展方向，即垂直优化、水平扩展和RAG生态系统。

8.1 RAG的垂直优化

尽管过去一年RAG技术取得了迅速进展，但在其垂直领域仍有几个需要进一步研究的领域。

1. **长上下文问题**：
 - 如文献[Xu et al., 2023c]所述，RAG的生成阶段受限于LLMs的上下文窗口。如果窗口太短，可能不包含足够的相关信息；如果太长，可能导致信息丢失。目前，扩展LLMs的上下文窗口，甚至达到无限制上下文，是LLM发展的一个关键方向。然而，一旦移除上下文窗口限制，RAG应如何适应仍然是一个值得注意的问题。
2. **RAG的鲁棒性**：

- 如果检索过程中出现无关噪声，或检索内容与事实相矛盾，可能会显著影响RAG的有效性。因此，增强RAG的鲁棒性越来越受到研究者的关注，如[Yu et al., 2023a, Glass et al., 2021, Baek et al., 2023]所示。

3. RAG与微调的协同：

- 混合方法已逐渐成为RAG的主流方法之一，如RADIT [Lin et al., 2023]所示。如何协调二者之间的关系，同时获得参数化和非参数化的优势，是一个需要解决的问题。

4. RAG的工程实践：

- RAG的实施便利性和与企业工程需求的一致性促成了其崛起。然而，在工程实践中，如何提高大规模知识库场景中的检索效率和文档回调率，以及如何确保企业数据安全（例如防止LLMs泄露文档的来源、元数据或其他信息）是需要解决的关键问题[Alon et al., 2022]。

8.2 水平扩展的RAG

RAG的研究在水平领域迅速扩展。从最初的文本问答领域开始，RAG的理念已逐渐应用于更多模态数据，如图像、代码、结构化知识、音频和视频等。在这方面已经有了许多工作。

1. 图像领域：

- BLIP-2[Li et al., 2023a]的提出使用冻结的图像编码器和大型语言模型进行视觉语言预训练，降低了模型训练成本。此外，该模型可以从零样本生成图像到文本的转换。

2. 代码领域：

- RBPS[Nashid et al., 2023]用于与代码相关的小规模学习。通过编码或频率分析，自动检索与开发者任务类似的代码示例。该技术已在测试断言生成和程序修复任务中证明了其有效性。

3. 结构化知识领域：

- 方法如CoK[Li et al., 2023c]首先从知识图谱中检索与输入问题相关的事实，然后以提示的形式将这些事实添加到输入中。这种方法在知识图谱问答任务中表现良好。

4. 音频和视频领域：

GSS[Zhao et al., 2022]方法检索并串联来自口语词汇库的音频片段，立即将MT（机器翻译）数据转换为ST（语音翻译）数据。UEOP[Chan et al., 2023]在端到端自动语音识别中引入了新的突破，通过引入外部离线策略进行声音到文本的映射。Vid2Seq[Yang et al., 2023a]架构通过引入特殊的时间标记，使语言模型能够在同一输出序列中无缝预测事件边界和文本描述。

8.3 RAG 生态

RAG（检索-生成增强）生态系统的发展涉及到下游任务的应用、评估体系的完善，以及相关技术栈的发展。

8.3.1 下游任务与评估

RAG通过整合广泛知识库中的相关信息，展示了在处理复杂查询和生成信息丰富响应方面的显著潜力。众多研究表明，RAG在各种下游任务中表现良好，例如开放式问题回答和事实验证。RAG模型不仅提高了下游应用中信息的准确性和相关性，而且增加了响应的多样性和深度。

鉴于RAG的成功，未来的工作将探索该模型在多领域应用中的适应性和普遍性。这包括其在专业领域知识问答中的应用，如医学、法律和教育等。在专业领域知识问答等下游任务的应用中，RAG可能提供比微调更低的训练成本和更好的性能优势。

同时，改进RAG的评估体系，以评估和优化其在不同下游任务中的应用，对于模型在特定任务中的效率和效益至关重要。这包括为不同下游任务开发更准确的评估指标和框架，如上下文相关性、内容创造性和无害性等。

此外，通过RAG增强模型的可解释性，使用户更好地理解模型如何以及为何做出特定响应，也是一项有意义的任务。

8.3.2 技术栈

在RAG的生态系统中，相关技术栈的发展发挥了推动作用。例如，随着ChatGPT的普及，LangChain和LLamaIndex迅速成为知名的RAG相关API提供者，逐渐成为大模型时代不可或缺的技术之一。同时，不断有新型技术栈被开发。虽然这些技术栈不像LangChain和LLamaIndex提供那么多功能，但更注重其独特特性。例如，Flowise AI 强调低代码，允许用户通过拖放操作实现各种AI应用，包括RAG，无需编写代码。其他新兴技术包括HayStack、Meltano和Cohere Coral。

除了AI原生框架，传统软件或云服务提供商也扩展了他们的服务范围。例如，向量数据库公司Weaviate提供的Verba 专注于个人助理。亚马逊提供了基于RAG思想的智能企业搜索服务工具Kendra，用户可以通过内置连接器在不同的内容存储库中搜索。

技术栈的发展与RAG相互增强。新技术对现有技术栈提出了更高的要求，而技术栈功能的优化进一步促进了RAG技术的发展。总的来说，RAG工具链的技术栈已初步形成，许多企业级应用逐渐出现，但全能型平台仍需进一步完善。

9 Conclusion

这篇论文全面探讨了检索增强生成（RAG），这是一种使用外部知识库来补充大型语言模型（LLMs）上下文并生成响应的技术。值得注意的是，RAG结合了LLMs的参数化知识和非参数化的外部知识，缓解了幻觉问题，通过检索技术及时识别信息，并提高了响应的准确性。此外，通过引用来源，RAG提高了模型输出的透明度和用户信任。通过索引相关文本语料库，RAG还可以根据特定领域进行定制。RAG的发展和特点被总结为三种范式：Naive RAG、Advanced RAG和Modular RAG，每种范式都有其模型、方法和缺点。Naive RAG主要涉及“检索-阅读”过程。Advanced RAG使用更精细的数据处理，优化知识库索引，并引入多次或迭代检索。随着探索的深入，RAG融合了其他技术，如微调，导致Modular RAG范式的出现，该范式通过新模块丰富了RAG过程，并提供了更多的灵活性。